ПЗ 5.2 Использование компилятора Numba для генерации кода под GPU

Методические рекомендации

Для инспекции имеющейся инфраструктуры (возможности Numba) и получения отчета об аппаратном обеспечении в терминале необходимо выполнить следующую ко

numba –s

Необходимо получить отчет по аппаратному обеспечению:

```
__LLVM Information__
LLVM Version                                : 14.0.6

__CUDA Information__
CUDA Device Initialized                     : True
CUDA Driver Version                         : 11.7
CUDA Runtime Version                        : 11.8
CUDA NVIDIA Bindings Available              : False
CUDA NVIDIA Bindings In Use                 : False
CUDA Minor Version Compatibility Available  : False
CUDA Minor Version Compatibility Needed     : True
CUDA Minor Version Compatibility In Use     : False
CUDA Detect Output:
Found 1 CUDA devices
id 0    b'NVIDIA GeForce MX230'                                 [SUPPORTED]
                      Compute Capability: 6.1
                           PCI Device ID: 0
                              PCI Bus ID: 2
                                    UUID: GPU-6090d386-3443-5ba1-09ac-0d4971729eed
                                Watchdog: Enabled
                            Compute Mode: WDDM
              FP32/FP64 Performance Ratio: 32
Summary:
       1/1 devices are supported
```

Также необходимо убедиться, что все библиотеки обнаружены:

```
CUDA Libraries Test Output:
Finding driver from candidates:
        nvcuda.dll
        \windows\system32\nvcuda.dll
Using loader <class 'ctypes.WinDLL'>
        Trying to load driver...        ok
                Loaded from nvcuda.dll
Finding nvvm from Conda environment
        Located at C:\Users\vadid\anaconda3\Library\bin\nvvm64_40_0.dll
        Trying to open library...       ok
Finding nvrtc from Conda environment
        Located at C:\Users\vadid\anaconda3\Library\bin\nvrtc64_112_0.dll
        Trying to open library...       ok
Finding cudart from Conda environment
        Located at C:\Users\vadid\anaconda3\Library\bin\cudart64_110.dll
        Trying to open library...       ok
Finding cudadevrt from Conda environment
        Located at C:\Users\vadid\anaconda3\Library\bin\cudadevrt.lib
        Checking library...     ok
Finding libdevice from Conda environment
        Located at C:\Users\vadid\anaconda3\Library\bin\libdevice.10.bc
        Checking library...     ok
```

Если требуемых библиотек нет, то их необходимо установить, в частности библиотеку cudatoolkit.

1. Имеется массив из 100 элементов. Ставится задача построить массив с удвоенным значением из исходного:

```
import numpy as np


a = np.ones(100)

b = np.empty(100)

for i in range(100):

   b[i] = 2 * a[i]
```

При осуществлении вычислений на центральном процессоре соответствующая функция просто последовательно проходит по массиву и удваивает значения.

При выполнении вычислений на графическом процессоре для каждого элемента будет создан отдельный поток. Код будет производить действие только с одним элементом.

Запустим первую версию кода:

```
from numba import cuda


@cuda.jit
def double(my_array):
   position = cuda.grid(1)
   my_array[position] *= 2
```

Никакого вывода у этой функции нет, потому что она будет выполняться графическим процессором. Функция не возвращает значения через return (не имеет вывода), потому что она предназначена для выполнения на GPU (графическом процессоре), где работа происходит непосредственно с данными в памяти, а не через возвращаемые значения.

Перед нашей функцией размещен декоратор @cuda.jit, который «просит» компилятор Numba сгенерировать версию нашего кода для CUDA. Функция cuda.grid - извлекает единственную позицию в массиве, значение которой необходимо изменять.

Еще раз отметим, что функция не может ничего возвращать, поскольку будет реализована в виде функции ядра GPU, так что для запуска вычислений необходимо передавать ей параметры для получения возвращаемых значений:

Получаем следующую ошибку:

```
Kernel launch configuration was not specified. Use the syntax:
kernel_function[blockspergrid, threadsperblock](arg0, arg1, ..., argn)
```

Данная ошибка возникла, потому что не указана конфигурация запуска ядра (количество блоков и потоков). В случае с GPU вычисления разбиваются на блоки, а каждый блок на потоки.

Запустим вариант с разбиением кода:

```
#вариант с разбиением
import numpy as np

blocks_per_grid = 50
threads_per_block = 20

my_array = np.ones(1000)
double[blocks_per_grid, threads_per_block](my_array)  # правильный вызов
assert (my_array == 2).all() #Проверка на то, что функция была применена ко всем элементам в массиве
```

В данном коде:

- blocks_per_grid = 50 - количество блоков в сетке
- threads_per_block = 20 - количество потоков в блоке
- Общее количество потоков: 50 × 20 = 1000, что соответствует размеру массива
- Утверждение assert проверяет, что все элементы массива стали равны 2

!!! Возможно, в ходе выполнения кода потребуется обновить драйверы NVIDIA на официальном сайте или обновить версию Numba. Также возможно потребуется явное указание совместимых версий.

В ходе выполнения данного кода может возникнуть масса ошибок. Необходимо выполнить следующую диагностику на предмет выявления ошибок:

```
import subprocess
import sys
```

```
def check_system():
    print("=== ДИАГНОСТИКА СИСТЕМЫ ===")

    # Проверка версий
    import numba
    print(f"Numba: {numba.__version__}")
    print(f"Python: {sys.version}")

    # Проверка CUDA через nvidia-smi
    try:
        result = subprocess.run(['nvidia-smi'], capture_output=True, text=True)
        if result.returncode == 0:
            print("nvidia-smi выполнен успешно")
            lines = result.stdout.split('\n')
            for line in lines[:5]:
                print(line)
        else:
            print("nvidia-smi не доступен")
    except:
        print("nvidia-smi не найден")

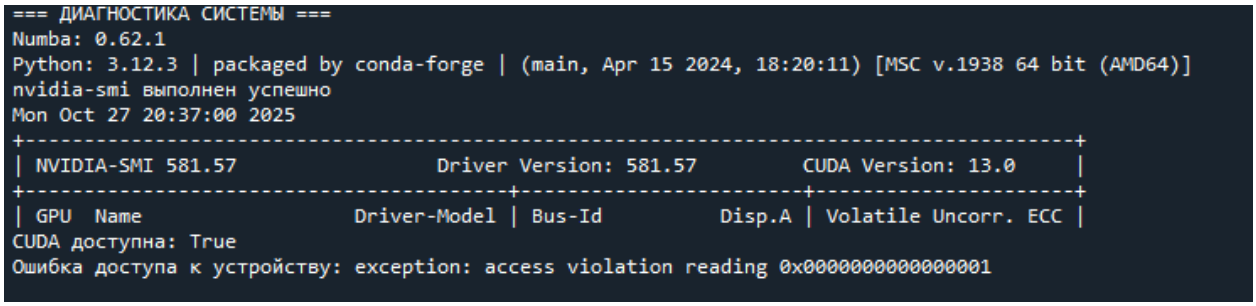
    # Проверка через numba
    from numba import cuda
    print(f"CUDA доступна: {cuda.is_available()}")

    if cuda.is_available():
```

```
    try:
        device = cuda.get_current_device()
        print(f"Устройство: {device.name}")
        print(f"Compute Capability: {device.compute_capability}")
    except Exception as e:
        print(f"Ошибка доступа к устройству: {e}")


check_system()
```

Возможный результат:



Продолжим.

Иногда невозможно получить точное количество элементов в массиве путем перемножения количества блоков и потоков - например, если количество значений в массиве соответствует простому числу. В таких случаях необходимо убедиться, что произведение количества блоков и потоков будет превышать количество элементов в обрабатываемом массиве.

```
threads_per_block = 16

blocks_per_grid = 63

my_array = np.ones(1000)

double[blocks_per_grid, threads_per_block](my_array)

assert (my_array == 2).all()
```

Расчет:

-Потоков в блоке: 16

-Блоков в сетке: 63

-Общее количество потоков: 16 × 63 = 1008

-Размер массива: 1000

Поскольку 1008 > 1000, некоторые потоки будут «лишними»

Исправление возникшей ситуации:

```
@cuda.jit
def double_safe(my_array):
    position = cuda.grid(1)
    if position > my_array.shape[0]:
        return
    my_array[position] *= 2
```

Далее запускаем код:

```
my_array = np.ones(1000)
double_safe[blocks_per_grid, threads_per_block](my_array)
assert (my_array == 2).all()
```

Теперь давайте вернемся к магической функции cuda.grid, позволяющей определить позицию для вычисления. Мы постараемся понять, как она работает, путем явного обращения к некоторым полезным свойствам. Иногда есть необходимость все держать под контролем и осуществлять доступ к нужным атрибутам напрямую, например при работе с массивами, содержащими более трех измерений:

Вызов в потоке позволяет получить доступ к номеру блока и потока, в которых выполняется код. Также можно узнать размер блока. Свойство *cuda.blockIdx* содержит индекс блока, в котором запущен текущий поток. Свойство *cuda.blockDim* дает доступ к размеру текущего блока, а *cuda.threadIdx* – к индексу потока внутри блока. Обладая всей этой информацией, вы можете с уверенностью сказать, что каждый поток адресует свою позицию в массиве.

```
@cuda.jit
def double_safe_explicit(my_array):
    position = cuda.blockIdx.x * cuda.blockDim.x + cuda.threadIdx.x
    if position >= my_array.shape[0]:
        return
    my_array[position] *= 2
```

Информация о расположении элемента для потока может быть одно-, дву- и трехмерной. Вероятно, вы заметили, что во всех трех вызовах в предыдущем коде мы обращались к параметру .x. Если вы имеете дело в двумерными или трехмерными массивами, можете воспользоваться параметрами .y и .z.

Теперь давайте взглянем на ту же функцию, способную работать с двумерными массивами:

```
@cuda.jit
def double_matrix_unsafe(my_matrix):
    x, y = cuda.grid(2)
    my_matrix[y, x] *= 2
```

Здесь мы вызываем функцию cuda.grid(2) для получения двух индексов. Обратите внимание, что мы вернулись к небезопасному коду, поскольку можем быть уверены, что у нас возникнет ошибка. Давайте запустим приведенный ниже код:

```
threads_per_block_2d = 16, 16
blocks_per_grid_2d = 63, 63
my_matrix = np.ones((1000, 1000))
double_matrix_unsafe[blocks_per_grid_2d, threads_per_block_2d](my_matrix)
print((my_matrix == 2).all())
```

Результатом выполнения этого кода будет вывод значения True, поскольку все элементы массива теперь содержат значение 2. Обратите внимание, что определения блоков и потоков стали двумерными, как и наш массив данных.

Если вы запустите этот код и вам повезет, то ошибка не появится. Но и результат вы наверняка увидите неправильный. Причина этого в том, что мы не выполняем проверку на границы массивов, и при выходе за границы одной строки переходим на следующую, что было невозможно в случае с одномерным массивом. Таким образом, в определенных позициях в массиве у нас могут оказаться не двойки, а четверки, поскольку код в них будет выполняться дважды.

Исправить это довольно просто. Ниже представлена окончательная версия с явным индексированием массивов:

```
@cuda.jit
def double_matrix(my_matrix):
    x = cuda.blockIdx.x * cuda.blockDim.x + cuda.threadIdx.x
    y = cuda.blockIdx.y * cuda.blockDim.y + cuda.threadIdx.y
    if x >= my_matrix.shape[0]:
        return
    if y >= my_matrix.shape[1]:
        return
    my_matrix[y, x] *= 2
```

Далее воссоздадим генератор Мандельброта с использованием GPU.

2. Создание генератора Мандельброта с помощью GPU

Реализуем функцию Мандельброта для расчета значений в конкретной точке.

```
from numba import cuda
@cuda.jit(device=True)
```

```
def compute_point(c):
    i = -1
    z = complex(0, 0)
    while abs(z) < 2:
        i += 1
        if i == 255:
            break
        z = z**2 + c
    return 255 - (255 * i)
```

Обратите внимание, что к декоратору @cuda.jit мы добавили параметр device=True. Таким образом мы говорим компилятору Numba о том, что функция должна быть вызвана изнутри устройства. Функция устройства, в отличие от функции ядра, может возвращать значения.

Теперь давайте реализуем первую версию вызывающего кода – внешне логичную, но ошибочную:

```
@cuda.jit
def compute_all_points_doesnt_work(start, end, size, img_array):
    x, y = cuda.grid(2)
    if x >= img_array.shape[0] or y >= img_array.shape[1]:
        return
    mandel_x = (end[0] - start[0]) * (x / size) + start[0]
    mandel_y = (end[1] - start[1]) * (y / size) + start[1]
    img_array[y, x] = compute_point(complex(mandel_x, mandel_y))
```

Несмотря на успешную компиляцию этого кода, при попытке его вызова вы получите ошибку.

Проблема в том, что компилятор Numba не умеет обрабатывать в качестве входных параметров кортежи (по крайней мере пока). Это отсылает нас к более

пространным размышлениям о том, что далеко не весь функционал, реализованный в Python, поддерживается в Numba. У вас всегда должна быть под рукой документация по этому компилятору, расположенная по адресу http://numba.pydata.org, чтобы можно было проверять, поддерживаются ли те или иные функции.

Необходимо избавится в нашем решении от кортежей в качестве входных параметров:

```
@cuda.jit
def compute_all_points(startx, starty, endx, endy, size, img_array):
    x, y = cuda.grid(2)
    if x >= img_array.shape[0] or y >= img_array.shape[1]:
        return
    mandel_x = (endx - startx) * (x / size) + startx
    mandel_y = (endy - starty) * (y / size) + starty
    img_array[y, x] = compute_point(complex(mandel_x, mandel_y))
```

Далее вызовем функцию:

```
from math import ceil
import numpy as np
from PIL import Image

size = 2000
start = -1.5, -1.3
end = 0.5, 1.3

img_array = np.empty((size, size), dtype=np.uint8)
threads_per_block_2d = 16, 16
blocks_per_grid_2d = ceil(size / 16), ceil(size / 16)
```

```
compute_all_points[blocks_per_grid_2d,
    threads_per_block_2d](start[0], start[1], end[0], end[1],
    size, img_array)


img = Image.fromarray(img_array, mode="P")
img.save("mandelbrot.png")
```

Обратим внимание на то, как объявляется переменная для хранения количества блоков. С учетом того, что мы используем 16 потоков на каждом блоке для каждого измерения, размер мы должны вычислять как size/16 с округлением результата в большую сторону.

Выполнение замера времени для этого вызова:

```
%timeit compute_all_points[blocks_per_grid_2d, threads_per_block_2d]
```

Скорость выполнения гораздо выше скорости из предыдущего задания. Конечно, силы медленного CPU и быстрого GPU изначально не равны. Более того, здесь есть и другие факторы, такие как используемый алгоритм и необходимость передачи данных между центральным и графическим процессором, оказывающие существенное влияние на быстродействие. Несмотря на это, в целом должно быть понятно, что некоторые алгоритмы при запуске на графическом процессоре работают быстрее, чем на центральном.

Таким образом, мы создали первый генератор Мандельброта с использованием GPU и получили значительную прибавку в скорости.